



UNIVERSITY OF ESWATINI

CSC392: Principles of Software Engineering I — Mini Project

SOFTWARE REQUIREMENTS SPECIFICATION

Universal Exam Card System

Field	Detail
Document Version	1.2 (24 May 2026)
Date	17 May 2026
Prepared by	Requirements Analyst
Approved by	Team Leader
Status	Baseline
Institution	University of Eswatini
Course	CSC392 — Principles of Software Engineering I

Table of Contents

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, Acronyms, and Abbreviations
 - 1.4 References
 - 1.5 Overview
- 2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Functions
 - 2.3 User Characteristics
 - 2.4 Constraints
 - 2.5 Assumptions and Dependencies
 - 2.6 Apportioning of Requirements
- 3. Specific Requirements
 - 3.1 External Interface Requirements
 - 3.1.1 System Interfaces
 - 3.1.2 User Interfaces
 - 3.1.3 Hardware Interfaces
 - 3.1.4 Software Interfaces
 - 3.1.5 Communications Interfaces
 - 3.2 Functional Requirements
 - 3.3 Performance Requirements
 - 3.4 Logical Database Requirements
 - 3.5 Design Constraints
 - 3.6 Software System Attributes
- 4. System Models
 - 4.1 Entity-Relationship Diagram
 - 4.2 Data Flow Diagrams
 - 4.3 Use Case Diagram
 - 4.4 Detailed Use Cases
- Appendix A — Data Dictionary
- Appendix B — Requirements Traceability Matrix
- Appendix C — Risks and Mitigation

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for the Universal Exam Card System — a secure digital solution designed to automate student identity verification and examination script tracking at the University of Eswatini.

This document is intended for the following audiences:

- Software developers responsible for design and implementation.
- System architects designing the backend, mobile, and web components.
- Quality assurance engineers developing test plans and validation criteria.
- University administrators and project stakeholders reviewing system scope.
- The project Team Leader who must formally approve this baseline.

1.2 Scope

The system to be developed is the Universal Exam Card System (UECS), hereafter referred to as "the system." The system replaces the University's current paper-based exam identity verification and script collection processes with an integrated digital solution.

In Scope:

- Generation of a digital Universal Exam Card embedding a unique QR code per student, consolidating all registered modules for a given examination period.
- Real-time QR code scanning and identity verification via an Android mobile application.
- Automated attendance marking and script collection logging linked to verified student records.
- Role-based web dashboard for administrators and lead invigilators, providing real-time monitoring, anomaly detection, and reporting.
- RESTful backend API developed in Node.js/Express with a MySQL relational database.
- Offline-capable Android application with local-storage buffering and automatic synchronisation when connectivity is restored.

Out of Scope:

- Physical exam card printing and physical distribution logistics.
- Biometric authentication (fingerprint, facial recognition).
- Integration with existing University financial or Human Resources information systems.
- Online exam proctoring or remote invigilation functionality.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
Universal Exam Card (UEC)	A single digital identity token (QR code) representing all of a student's registered examination modules for a given period.

Term / Acronym	Definition
QR Code	Quick Response code — a two-dimensional barcode encoding student verification data.
Scan Log	A timestamped record of a verification or script-collection event stored in the database.
Invigilator	An examination supervisor authorised to scan students and log scripts.
Lead Invigilator	A senior supervisor with access to the real-time web dashboard.
JWT	JSON Web Token — a signed token used for stateless session authentication.
API	Application Programming Interface — the RESTful service layer exposed by the backend.
SRS	Software Requirements Specification — this document.
FR	Functional Requirement.
NFR	Non-Functional Requirement.
UC	Use Case.
UECS	Universal Exam Card System — the system being specified.
AWS	Amazon Web Services — the target cloud deployment platform.
TBD	To Be Determined — items requiring resolution before the next SRS revision.

1.4 References

The following documents and standards are referenced within this SRS:

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications. Institute of Electrical and Electronics Engineers, 1998.
- IEEE Std 610.12-1990, IEEE Standard Glossary of Software Engineering Terminology.
- IEEE Std 1074-1997, IEEE Standard for Developing Software Life Cycle Processes.
- University of Eswatini Academic Examination Regulations, 2024 Edition.
- OWASP Top Ten Security Risks, 2021 Edition — <https://owasp.org/www-project-top-ten/>
- Node.js Documentation (v20 LTS) — <https://nodejs.org/docs/>
- React.js Documentation (v18) — <https://react.dev/>
- Android Developer Documentation (Java) — <https://developer.android.com/docs>

1.5 Overview

The remainder of this SRS is organised as follows:

- Section 2 — Overall Description: Provides context for the system, including product perspective, high-level functions, user characteristics, constraints, assumptions, and phasing of requirements.

- Section 3 — Specific Requirements: Contains all detailed functional and non-functional requirements, organised by category, each uniquely identified and verifiable.
- Section 4 — System Models: Contains the Entity-Relationship Diagram, Data Flow Diagrams (Context and Level 1), and Use Case Diagram, together with detailed use case descriptions.
- Appendix A — Data Dictionary: Defines all database entities and their attributes.
- Appendix B — Requirements Traceability Matrix: Maps each functional requirement to use cases, project objectives, and priority levels.
- Appendix C — Risks and Mitigation: Documents identified project risks and corresponding mitigation strategies.

2. Overall Description

2.1 Product Perspective

The UECS is a new, self-contained system that replaces existing manual examination processes. Although independent, it must operate within the University's existing IT environment, which includes the student registration database and the University network infrastructure.

The system comprises three primary subsystems:

Subsystem	Technology	Role
RESTful Backend API	Node.js / Express	Central business logic, authentication, and data access layer.
Web Dashboard	React.js	Role-based monitoring and reporting interface for administrators and lead invigilators.
Mobile Application	Android (Java)	QR code scanning, identity verification, and offline-capable data collection for invigilators.
Relational Database	MySQL	Persistent storage for student records, exam schedules, and scan logs.

System Interfaces:

- The backend API communicates with the MySQL database via a connection pool.
- The React web dashboard consumes the backend API over HTTPS (REST/JSON).
- The Android app communicates with the backend API over HTTPS; locally buffers data using SQLite when offline.

User Interfaces:

- Web dashboard: browser-based, responsive layout supporting Chrome, Firefox, and Edge.
- Android app: native UI conforming to Material Design guidelines, operable on Android 8.0 (API 26) and above.

Hardware Interfaces:

- Android device camera (minimum 5 MP autofocus) for QR code scanning.
- Server hardware (AWS EC2 instance or equivalent) hosting the backend and database.

Software Interfaces:

- Student registration data shall be imported as a periodic CSV export from the University's existing student information system (no live integration in v1.0 — see Section 2.6).

Communications Interfaces:

- HTTPS (TLS 1.2 or higher) for all client-server communication.

- Wi-Fi connectivity assumed within examination venues; offline mode activates automatically on loss of connectivity.

Memory Constraints:

- Android app local SQLite cache shall not exceed 100 MB on-device storage.

2.2 Product Functions

The following high-level functions summarise the capabilities of the UECS:

Function ID	High-Level Function	Primary Actor
F-01	Universal Exam Card Generation	System / Administrator
F-02	User Authentication and Session Management	Student, Invigilator, Administrator
F-03	QR Code Identity Verification	Invigilator
F-04	Automated Attendance Marking	System
F-05	Script Collection Logging	Invigilator
F-06	Real-Time Dashboard and Monitoring	Lead Invigilator, Administrator
F-07	Report Generation	Administrator

2.3 User Characteristics

User Class	Description	Technical Expertise	Access Level
Student	Registered University students sitting examinations. Expected to access the system solely to view their digital exam card and schedule.	Basic smartphone literacy assumed.	Read-only — own records only.
Invigilator	Examination supervisors operating during exam sessions. Expected to use the Android app under time pressure.	Basic mobile app proficiency; no IT specialisation required. Must complete a 30-minute onboarding session.	Operational — scanning and logging.
Lead Invigilator / Administrator	Senior supervisors or academic administration staff responsible for exam integrity oversight and reporting.	Moderate IT literacy; familiarity with web-based dashboards.	Full access including scheduling, reporting, and user management.

2.4 Constraints

This section documents constraints on the developer's design options as defined by IEEE Std 830-1998 §5.2.4. Project-level constraints (schedule, budget, team organisation) are documented in the CSC392 Mini Project Plan and should be consulted alongside this SRS. Each constraint below is a hard limit on valid design choices, not a preference or recommendation.

C-01 Regulatory Policies

The system shall comply with the University of Eswatini's data protection and academic integrity regulations. Any feature that stores, transmits, or displays personally identifiable student data must conform to those policies. Non-compliance is not a design option.

C-02 Mandatory Technology Stack

The implementation shall use the following technologies without substitution: Node.js/Express (backend API), React.js (web dashboard), Java (Android mobile app), and MySQL 8.0+ (relational database). These are mandatory institutional requirements for this project — not recommendations. Any deviation requires prior written approval from the Team Leader before work begins.

C-03 Cloud Deployment Platform

The system shall be designed for deployment exclusively on Amazon Web Services (AWS). The design must not rely on features exclusive to any other cloud provider. Target services: AWS EC2 (backend), AWS RDS (database), AWS Amplify or S3 (web frontend).

C-04 Network Availability

The system shall not assume continuous network connectivity within examination venues. The Android application must operate fully offline and synchronise automatically when connectivity is restored. Any architecture that requires a persistent connection to function correctly is non-compliant with this constraint.

C-05 Security Controls (Non-Negotiable)

The following controls are non-negotiable and not subject to trade-offs: (a) all data in transit shall be encrypted via TLS 1.2 or higher; (b) passwords shall be stored only as bcrypt hashes, minimum cost factor 12; (c) all database queries shall use parameterised statements; (d) all API endpoints except the login endpoint shall require a valid JWT.

C-06 Audit and Integrity Logging

The system shall maintain an immutable audit trail of all identity-verification and script-collection events in the scan_logs table. No design may permit deletion or retroactive modification of existing log records. This is a regulatory requirement tied to the University's examination integrity policies.

2.5 Assumptions and Dependencies

Assumptions:

- Reliable Wi-Fi infrastructure is available in designated examination venues; should this assumption prove false, offline synchronisation provides adequate fallback.
- Students have access to a smartphone (Android or iOS) capable of displaying QR codes; alternatively, the University will provide a printed fallback procedure.
- Invigilators will carry Android devices (API 26+) provided or approved by the University.
- A periodic CSV export of student registration data from the existing student information system will be available to seed the UECS database.
- Each examination session has a pre-defined venue, date, time, and enrolled student list loaded into the system before the session begins.

Dependencies:

- Correct and timely student registration data from the academic registry.
- AWS account provisioning and appropriate IAM permissions for deployment.
- Android device availability for invigilators prior to the pilot examination period.

2.6 Evolution of Requirements

Future enhancements such as live SIS API integration, biometric verification, iOS support, and other advanced capabilities are anticipated but are not part of the v1.0 baseline. These enhancements will be documented in subsequent revisions once validated through prototype testing, stakeholder review, and formal change management procedures.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 System Interfaces

- SI-SYS-01: The UECS shall interface with the University's existing Student Information System (SIS) via a periodic CSV data export. The UECS backend shall provide an import endpoint that accepts a UTF-8 encoded CSV file conforming to an agreed schema (student_id, name, email, registered_modules, examination_period). Live or real-time API integration with the SIS is deferred to v2.0 (see Section 2.6).
- SI-SYS-02: The UECS shall not write data back to the University SIS in v1.0. All data originating from the SIS is treated as read-only reference data once imported.
- SI-SYS-03: The CSV import interface shall validate incoming records against defined schema rules and reject malformed rows with a structured error report, without aborting the full import.

3.1.2 User Interfaces

- UI-01: The web dashboard shall display real-time attendance data and refresh at intervals of no greater than 5 seconds without requiring a manual page reload.
- UI-02: The Android app QR scan interface shall provide a clearly visible viewfinder overlay and audible and visual feedback (green checkmark for success, red cross for failure) within 2 seconds of scan completion.
- UI-03: Error messages in the Android app shall be concise (maximum 15 words), written in plain English, and displayed for a minimum of 3 seconds.
- UI-04: The web dashboard and mobile app shall support the English language; Siswati localisation is deferred to v2.0.

3.1.3 Hardware Interfaces

- HI-01: The Android application shall interface with the device camera using the Android Camera2 API or the CameraX library for QR code image capture.
- HI-02: The backend server shall expose all services on standard HTTPS port 443.

3.1.4 Software Interfaces

- SI-01: The backend shall expose a RESTful API using JSON payloads over HTTPS conforming to REST architectural constraints (stateless, resource-oriented URIs).
- SI-02: The Android app shall use the ZXing or ML Kit Barcode Scanning library for QR code decoding.
- SI-03: The system shall use MySQL 8.0 or higher as its relational database management system.
- SI-04: JWT tokens shall conform to RFC 7519 and use the HS256 signing algorithm with a server-side secret.

SI-05 (External System Interface – SIS): The UECS shall import student registration data from the University SIS via periodic UTF-8 encoded CSV exports. Schema: (student_id, name, email, registered_modules, examination_period). Responsibility: University Registry provides the file;

UECS consumes it. Validation: Malformed rows shall be rejected with structured error reports without aborting the full import. Constraint: No live API integration in v1.0; deferred to future revisions. Directionality: Read-only reference data once imported; UECS shall not write back to SIS.

3.1.5 Communications Interfaces

- CI-01: All client-server communication shall use HTTPS with TLS 1.2 as a minimum.
- CI-02: The Android app shall detect loss of network connectivity and switch to offline mode transparently, queuing scan events in local SQLite storage for synchronisation.
- CI-03: Offline data queued by the Android app shall be synchronised to the backend within 60 seconds of network connectivity being re-established.

3.2 Functional Requirements

FR-01 — Universal Exam Card Generation

The system shall generate, for each enrolled student, a digital Universal Exam Card containing a unique, digitally signed QR code that encodes the student's ID, registered module codes, examination period, and a server-generated nonce. The QR code shall be regenerated for each examination period and invalidated after the period ends.

Priority: Critical

FR-02 — User Authentication

The system shall authenticate all users (students, invigilators, administrators) by email address and password. Upon successful authentication, the server shall issue a JWT with an expiry of no more than 8 hours. Passwords shall be validated against a bcrypt hash with a minimum cost factor of 12. After 5 consecutive failed login attempts, the account shall be temporarily locked for 15 minutes.

Priority: Critical

FR-03 — Identity Verification

The Android application shall scan a student's QR code and transmit the encoded payload to the backend API for verification. The backend shall verify (a) the QR code's digital signature, (b) the student's enrolment in the module scheduled at the scanned venue and time, and (c) that the student has not already been marked present. The verification result (pass or fail with reason) shall be returned to the app within 2 seconds under normal network conditions.

Priority: Critical

FR-04 — Automated Attendance Marking

Upon a successful identity verification (FR-03), the system shall automatically create a scan log entry with the student ID, invigilator ID, module code, venue, timestamp (UTC), and action type "verification". The system shall prevent duplicate attendance marking: a second scan of the same student for the same module session shall return an error and not create a duplicate log.

Priority: High

FR-05 — Script Collection Logging

The Android application shall provide an invigilator with the ability to confirm script submission for a previously verified student. Upon confirmation, the system shall create a scan log entry with action type "script_collection", linked to the original verification log. The system shall

prevent script collection being logged for a student who has not been marked present in the same session.

Priority: High

FR-06 — Real-Time Dashboard and Monitoring

The web dashboard shall provide authenticated lead invigilators and administrators with a real-time view of: (a) total students registered for each active session, (b) number and percentage verified, (c) number and percentage who have submitted scripts, and (d) any anomalies (e.g., students flagged as absent, duplicate scan attempts). Data shall be current to within 5 seconds.

Priority: High

FR-07 — Report Generation

The system shall allow administrators to generate and download reports in CSV and PDF formats covering: (a) attendance by session, (b) script collection by session, and (c) exam integrity summary (anomalies, flagged events). Reports shall be generated within 10 seconds for sessions containing up to 1,000 student records.

Priority: Medium

FR-08 – Session Expiry Security Control

The system shall automatically expire an invigilator's application session at the end of each examination session. After expiry, the invigilator shall be required to re-authenticate by submitting their email and password through the Android application login screen. The backend shall validate the credentials against the stored bcrypt hash and issue a new JWT that is valid only for the next scheduled examination session. Any offline-queued scans created after session expiry shall be rejected during synchronisation, and the invigilator shall not be permitted to perform any scanning or script-logging actions until a new valid JWT has been issued.

Priority: Critical

3.3 Performance Requirements

- PR-01: The backend API shall return identity verification responses (FR-03) within 2 seconds for 95% of requests under a load of up to 500 concurrent scans.
- PR-02: The system shall support a minimum of 500 simultaneous users across all client types (web dashboard and Android app combined).
- PR-03: The web dashboard shall load its initial view within 3 seconds on a connection of 10 Mbps or faster.
- PR-04: Report generation for sessions of up to 1,000 students shall complete within 10 seconds.
- PR-05: Offline scan events shall be synchronised to the backend within 60 seconds of network reconnection.

3.4 Logical Database Requirements

The database shall store the following categories of information:

- Student records: student_id, name, email (unique), hashed password, registration status.
- Module records: module_code (PK), module_name, credit_hours, semester.

- Enrollment records: student_id (FK), module_code (FK), examination period.
- Exam sessions: session_id (PK), module_code (FK), venue, date, start_time, end_time.
- Invigilator records: invigilator_id (PK), name, email, role (invigilator | lead_invigilator | administrator), hashed password.
- Universal Exam Cards: card_id (PK), student_id (FK), qr_payload (signed), valid_from, valid_until.
- Scan logs: log_id (PK), student_id (FK), invigilator_id (FK), session_id (FK), scan_time (TIMESTAMP UTC), action ENUM('verification', 'script_collection'), status ENUM('success', 'failure'), failure_reason (nullable).

Integrity constraints:

- A student may have at most one "verification" log per session (UNIQUE constraint on student_id + session_id + action = verification).
- A "script_collection" log must reference an existing "verification" log for the same student and session (enforced via application logic and FK constraints).
- Scan logs shall be retained for a minimum of 7 years in accordance with University academic records retention policy.

3.5 Design Constraints

3.5.1 Standards Compliance

- The system shall be developed in accordance with the team's agreed coding standards document (to be maintained in the project repository).
- API design shall follow RESTful conventions as described in RFC 7231.
- Security controls shall address the OWASP Top Ten (2021 edition) as a minimum baseline.

3.5.2 Technology Stack Constraints

- Backend: Node.js (v20 LTS) with Express framework.
- Web Frontend: React.js (v18).
- Mobile: Android Java targeting API level 26 (Android 8.0) as the minimum supported version.
- Database: MySQL 8.0 or higher.
- Deployment: AWS (EC2 for backend, RDS for database, S3 for static web assets).

3.6 Software System Attributes

3.6.1 Reliability

- The backend API shall have a target availability of 99.5% during scheduled examination periods (planned maintenance excluded).
- The Android app shall correctly handle all realistically expected network interruptions without data loss, by buffering events locally.

3.6.2 Availability

- The system shall be operational 24 hours a day, 7 days a week, with a maximum of 4 hours of planned maintenance downtime per month, preferably scheduled outside examination periods.

3.6.3 Security

- All passwords shall be stored as bcrypt hashes with a cost factor of 12 or higher.
- All API endpoints except the authentication endpoint shall require a valid JWT.
- All database queries shall use parameterised statements to prevent SQL injection.
- All data in transit shall be encrypted using TLS 1.2 or higher.
- Sensitive fields (e.g., QR code payload) shall be encrypted at rest using AES-256.
- QR codes shall include a server-signed digital signature to prevent forgery.

3.6.4 Maintainability

- The backend codebase shall be structured using the MVC (Model-View-Controller) pattern with clear separation of concerns.
- All modules shall include inline documentation comments (JSDoc for Node.js, Javadoc for Android).
- Unit test coverage shall be a minimum of 70% of business logic functions.

3.6.5 Portability

- The backend shall be containerisable using Docker and deployable to any standard Linux-based cloud environment without code changes.
- The web dashboard shall function correctly on Chrome (v110+), Firefox (v110+), and Edge (v110+).
- The Android app shall support Android 8.0 (API 26) through the current stable release.

3.6.6 Usability

- An invigilator with no prior training shall be able to complete a successful QR scan-and-verify sequence within 30 seconds after completing the 30-minute mandatory onboarding session.
- The mobile interface shall present no more than three interactive elements on any single screen to minimise cognitive load during high-pressure examination sessions.

4. System Models

This section presents all system models for the Universal Exam Card System. Two notation styles are used deliberately: structured analysis (Context Diagram and Level 1 DFD) and UML (Use Case Diagram and detailed use case tables). Per IEEE Std 830-1998 §5.3.8, multiple complementary notations are permitted and encouraged where each addresses a distinct modelling concern. DFDs describe how data flows across process and system boundaries; Use Case diagrams describe actor-system interactions at a functional level. Each model type answers a different question and the combination gives a more complete picture than either alone.

4.1 Entity-Relationship Diagram

Figure 4-1 illustrates the logical data model for the Universal Exam Card System, showing all primary entities, their attributes, and the relationships between them.

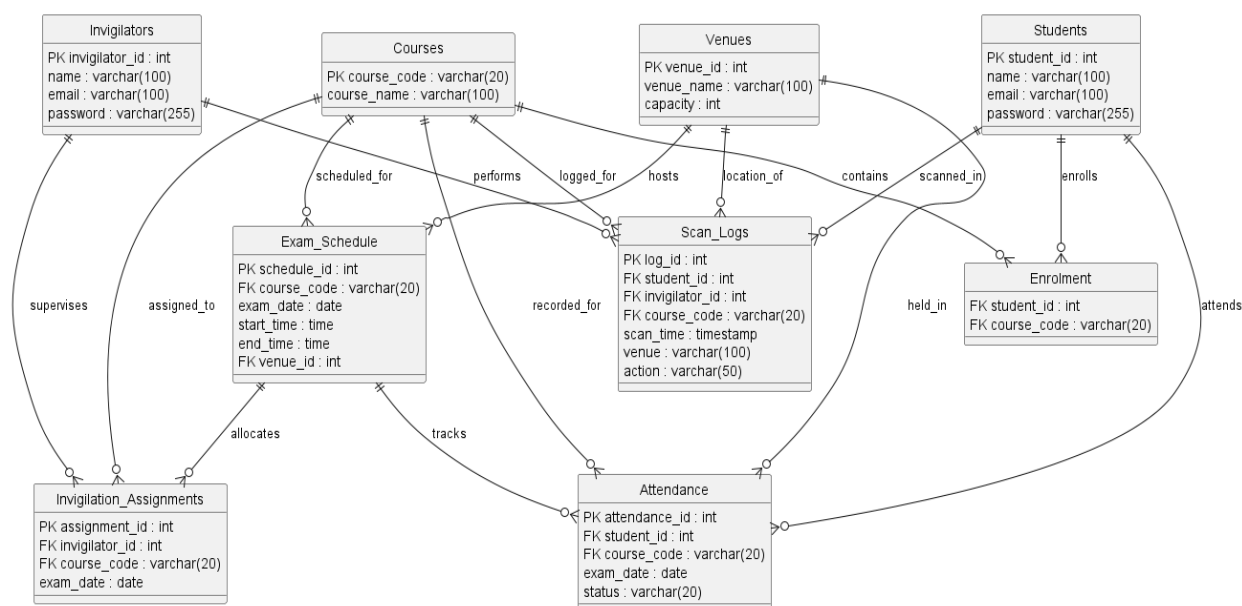


Figure 4-1: Entity-Relationship Diagram (ERD) — Universal Exam Card System

4.2 Data Flow Diagrams

4.2.1 Sequence Diagram

Figure 4-3 decomposes the system into its major processes, showing the data flows between processes and the data stores they access.

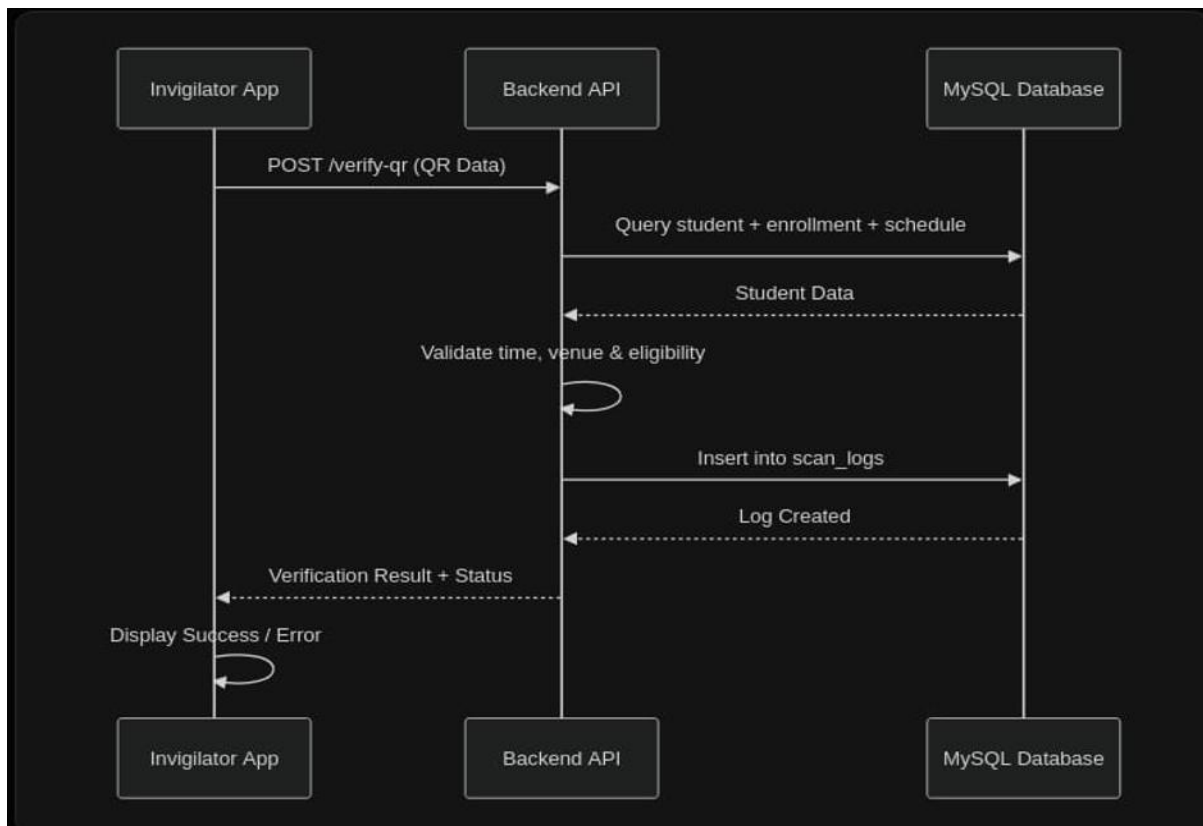


Figure 4-3: Sequence Diagram — Universal Exam Card System

4.3 Use Case Diagram

Figure 4-4 presents all use cases in scope for v1.0 and their associations with the identified actor classes.

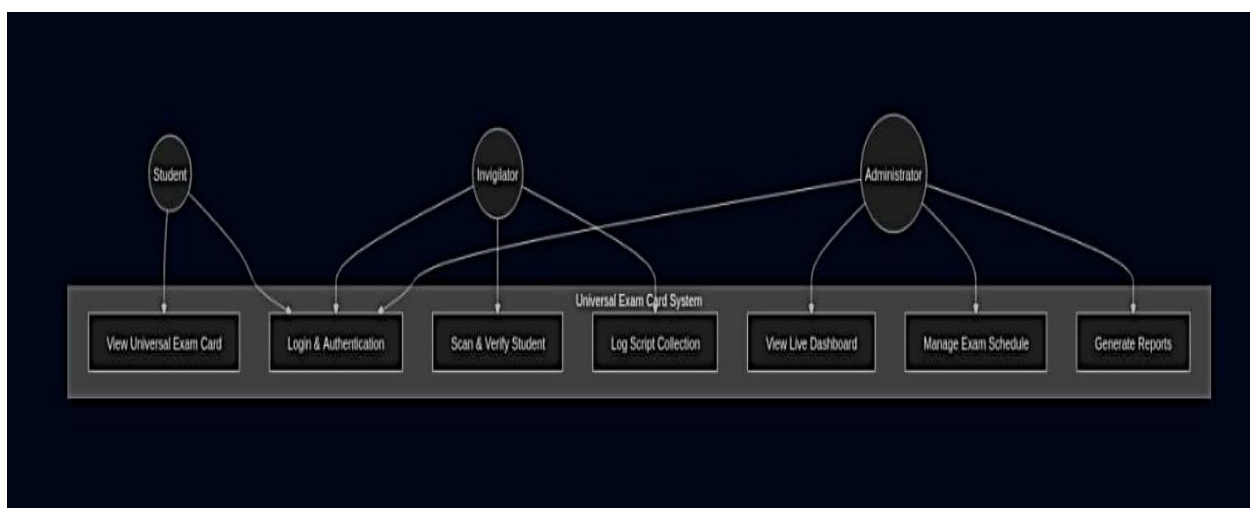


Figure 4-4: Use Case Diagram — Universal Exam Card System

4.4 Detailed Use Cases

UC-01: Generate Universal Exam Card

Field	Detail
Actor	System (scheduled trigger) / Administrator
Trigger	Start of a new examination period, or on-demand by Administrator
Pre-conditions	Student registration data loaded; examination schedule configured.
Main Success Flow	1. System retrieves all enrolled students for the period. 2. For each student, generates a signed QR payload. 3. Stores card record; makes card available to student via web/app login.
Alternative Flow	Administrator manually triggers regeneration for a specific student.
Exception Flows	Student not enrolled in any module → card not generated; administrator notified.
Post-conditions	Each eligible student has an active Universal Exam Card valid for the examination period.
Related FR	FR-01

UC-02: Authenticate User

Field	Detail
Actor	Student, Invigilator, or Administrator
Trigger	User submits email and password on the login screen.
Pre-conditions	User account exists in the system.
Main Success Flow	1. System validates email exists. 2. Compares submitted password with bcrypt hash. 3. Issues JWT; redirects to role-appropriate dashboard or exam card view.
Alternative Flow	N/A
Exception Flows	Invalid credentials → error message; 5th consecutive failure → account locked 15 minutes.
Post-conditions	Authenticated session established; JWT held by client.
Related FR	FR-02

UC-03: Scan and Verify Student

Field	Detail
Actor	Invigilator
Trigger	Student presents QR code at examination venue.

Field	Detail
Pre-conditions	Invigilator logged in; exam session active in the system.
Main Success Flow	1. Invigilator opens scanner in Android app. 2. Scans student's QR code. 3. App transmits QR payload to backend API. 4. Backend verifies signature, enrolment, schedule, and venue match. 5. Backend confirms student not already marked present. 6. System marks attendance and returns success response. 7. App displays green success indicator to invigilator.
Alternative Flow	Offline mode: scan event buffered in local SQLite; synchronised when connectivity restored (FR-04, CI-03).
Exception Flows	Invalid QR signature → error "Invalid exam card". Student not enrolled in session module → error "Not registered for this exam". Student already marked present → error "Already verified" (no duplicate log created).
Post-conditions	Scan log created; student attendance recorded in the database.
Related FR	FR-03, FR-04

UC-04: Log Script Collection

Field	Detail
Actor	Invigilator
Trigger	Student submits examination script at end of exam session.
Pre-conditions	Student has been previously verified present in this session (UC-03 completed successfully).
Main Success Flow	1. Invigilator selects student from the verified list in the Android app (or re-scans QR). 2. Invigilator taps "Confirm Script Received". 3. App sends script collection confirmation to backend. 4. Backend creates scan log with action = "script_collection", linked to original verification log. 5. App displays confirmation to invigilator.
Alternative Flow	Offline mode: event buffered locally and synchronised when connectivity is restored.
Exception Flows	Student not yet verified → error "Student not verified for this session; verify first". Script already logged → error "Script already recorded for this student".
Post-conditions	Script collection scan log created; dashboard reflects updated script count.
Related FR	FR-05

UC-05: Monitor Dashboard and View Reports

Field	Detail
Actor	Lead Invigilator, Administrator

Field	Detail
Trigger	Lead Invigilator or Administrator navigates to the dashboard or reports section.
Pre-conditions	Authenticated session with lead invigilator or administrator role.
Main Success Flow	1. Actor selects an active or completed examination session. 2. Dashboard displays real-time attendance and script submission metrics (counts, percentages, anomalies). 3. Data auto-refreshes every 5 seconds. 4. Actor optionally requests a report (attendance, script collection, or integrity summary). 5. System generates and provides a downloadable CSV or PDF report within 10 seconds.
Alternative Flow	Actor applies filters (venue, module, time range) to narrow dashboard view.
Exception Flows	Session data unavailable → display error message with retry option.
Post-conditions	Actor has current visibility of session metrics; optional report downloaded.
Related FR	FR-06, FR-07

Figure 4-5 provides a detailed use case flow diagram for UC-03 and UC-04, illustrating the interaction between the invigilator, the mobile app, and the backend API.

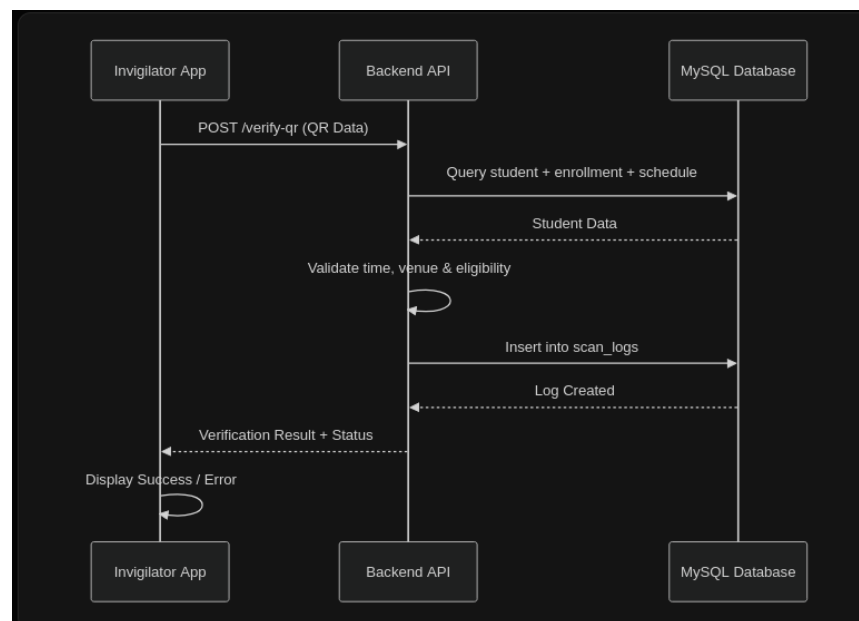


Figure 4-5: Detailed Use Case Flow — UC-03 Scan & Verify and UC-04 Log Script Collection

Appendix A — Data Dictionary

This appendix defines all principal database tables, their columns, data types, and integrity constraints. Additional tables should be added as the design is refined.

Table: students

Column	Type	Constraints	Description
student_id	INT	PK, AUTO_INCREMENT	Unique system-assigned student identifier.
name	VARCHAR(100)	NOT NULL	Full name of the student.
email	VARCHAR(100)	NOT NULL, UNIQUE	Institutional email address (login credential).
password	VARCHAR(255)	NOT NULL	bcrypt-hashed password (cost factor ≥ 12).
registration_status	ENUM	NOT NULL	Values: 'active' 'suspended' 'graduated'.

Table: scan_logs

Column	Type	Constraints	Description
log_id	INT	PK, AUTO_INCREMENT	Unique scan event identifier.
student_id	INT	FK → students	Student involved in the scan event.
invigilator_id	INT	FK → invigilators	Invigilator who performed the scan.
session_id	INT	FK → exam_sessions	Exam session in which the event occurred.
scan_time	TIMESTAMP	NOT NULL, DEFAULT UTC	UTC timestamp of the scan event.
action	ENUM	NOT NULL	Values: 'verification' 'script_collection'.
status	ENUM	NOT NULL	Values: 'success' 'failure'.
failure_reason	VARCHAR(255)	NULLABLE	Description of failure cause (null on success).

Note: Additional tables (modules, exam_sessions, invigilators, universal_exam_cards, enrollments) shall be documented in the database schema design artifact maintained in the project repository.

Appendix B — Requirements Traceability Matrix

The table below maps each functional requirement to its associated use case(s), project objective(s), IEEE 830-1998 characteristic, and priority classification.

FR ID	Requirement	Use Case(s)	Project Objective	Priority	Stability
FR-01	Universal Exam Card Generation	UC-01	Obj. 1 — Digital UEC	Critical	Stable
FR-02	User Authentication	UC-02	Obj. 4 — Security	Critical	Stable
FR-03	Identity Verification	UC-03	Obj. 2 — Secure Verification	Critical	Stable
FR-04	Automated Attendance Marking	UC-03	Obj. 2 — Verification / Obj. 3 — Logging	High	Stable
FR-05	Script Collection Logging	UC-04	Obj. 3 — Script Logging	High	Stable
FR-06	Real-Time Dashboard	UC-05	Obj. 4 — Invigilator Accountability	High	Stable
FR-07	Report Generation	UC-05	Obj. 4 — Invigilator Accountability	Medium	Conditional

Priority Definitions (per IEEE Std 830-1998 §4.3.5.2):

- Critical — System will not be acceptable unless this requirement is satisfied.
- High — Strongly enhances the system; absence would significantly impact stakeholder satisfaction.
- Medium — Desirable; absence would not make the system unacceptable for v1.0.

Appendix C — Risks and Mitigation

Risk ID	Risk Description	Likelihood	Impact	Mitigation Strategy
R-01	Network outage during exam sessions preventing real-time sync	Medium	High	Offline mode with local SQLite buffering and automatic sync (FR-04, CI-02, CI-03).
R-02	Security breach — unauthorised access to student data	Low	High	JWT authentication, bcrypt password hashing, TLS encryption, parameterised queries (FR-02, §3.6.3).
R-03	Data inconsistency between offline and online scan logs	Medium	High	Transaction management, duplicate-detection constraints, and atomic sync operations (FR-04).
R-04	Student registration data not available before exam period	Low	High	Early CSV export schedule agreed with academic registry; import tested in staging environment.
R-05	Android devices not available for all invigilators	Medium	Medium	University to procure and test devices prior to pilot; fallback to manual register if critical.
R-06	QR code forgery or duplication	Low	Critical	Server-signed QR payload with nonce; signature verified server-side on every scan (FR-01, FR-03).